aLiveHub: A Full-Stack SaaS Platform for Smart Sports Turf Booking in India

Saras Bari
Dept. of Information technology
Vidyavardhini's College of Engineering and Technology , *Mumbai, India*
sarasbari07@gmail.com

Abhay Barai
Dept. of Information technology.
Vidyavardhini's College of Engineering and Technology , *Mumbai, India*
baraiabhay3808@gmail.com

Prajval Chavan
Dept. of Information technology.
Vidyavardhini's College of Engineering and Technology , *Mumbai, India*
prajvalchavan1946@gmail.com

Anas Chauhan
Dept. of Information technology.
Vidyavardhini's College of Engineering and Technology , *Mumbai, India*
chauhananas348@gmail.com

Anish Bavishi
Dept. of Information technology.
Vidyavardhini's College of Engineering and Technology , *Mumbai, India*
anishbavishi15@gmail.com

Abstract—aLiveHub started as a direct response to a problem we kept running into ourselves: booking a sports turf in India is unnecessarily painful. You call, nobody answers. You WhatsApp, you wait. You show up and the slot is taken. We built this platform to fix that. aLiveHub is a full-stack SaaS system that brings players, turf owners, and admins onto one platform — with live slot availability, Razorpay payment integration, AI-assisted scheduling through Groq, and a Flutter mobile app for on-the-go bookings. The backend runs on Node.js and Express; the web frontend is built with React 18 and TypeScript. We chose Firebase Firestore as our primary database, layered Upstash Redis on top for caching, and used BullMQ to offload email delivery to background workers. Sentry keeps an eye on everything in production. This paper walks through how we designed and built the system, the decisions we made, and what we learned along the way.

Keywords—*SaaS, turf booking, React, Node.js, Firebase, Flutter, Razorpay, AI recommendations, real-time systems, slot management*

I. Introduction

aLiveHub is a full-stack sports turf booking platform built on a SaaS model. It bridges the gap between players who want to book a turf and owners who need a proper system to manage their business — offering turf discovery, real-time slot booking, secure payments, and an owner dashboard, all in one place.

The way sports turfs are booked in India today is honestly broken. It is mostly phone calls that go unanswered, WhatsApp threads that get ignored, and zero visibility into whether a slot is actually free. From the owner's side, things are just as messy — no dashboards, no revenue tracking, no way to grow. aLiveHub was built to address both sides of this problem through a three-sided platform that serves players, turf owners, and administrators at the same time.

On the technical side, we built around a React 18 frontend, a Node.js/Express REST API, and Firebase Firestore as our primary database, with a Flutter app handling mobile users. Razorpay takes care of payments, Groq drives the AI slot recommendations, BullMQ handles email delivery in the background, and Upstash Redis handles caching. This paper covers the full design, how the architecture comes together, and the key implementation choices we made.

II. Features

A. Player Experience

When a player lands on aLiveHub, they get a hero carousel on the home page, location-aware search, and AI-driven slot suggestions right up front. Finding a turf is straightforward — filter by sport, location, price, or date and the results update instantly. Each turf page has photo galleries, a list of amenities, user reviews, ground selection, and an interactive slot picker to choose exactly what you want.

Slot availability is live, and once you select a slot, it gets locked for 5 minutes so nobody else can grab it while you complete payment — that is how we prevent double bookings. Payments go through Razorpay, and every transaction is verified on the server using HMAC signature validation. Once a booking is confirmed, users get an animated success screen with confetti, a QR code for the booking, and an automated email. User profiles track booking history, saved favorites, and personal settings. Signing in just takes one click via Google Authentication through Firebase Auth.

B. Owner Dashboard

The owner dashboard gives turf owners visibility they have never had before — booking counts broken down by day, week, and month, revenue summaries, and occupancy rates at a glance. All bookings are listed with customer details and can be filtered by status. Owners can configure which time slots are available per ground, and update turf details, photos, pricing, amenities, and grounds without needing to contact anyone. Revenue data is displayed as interactive charts built with Recharts, and owners can export reports. They can also read and reply to customer

reviews, and manage notification preferences directly from the dashboard.

C. AI-Powered Features

We integrated Groq AI into the platform to make slot recommendations that actually feel relevant. Rather than showing generic popular times, the recommendation engine looks at a user's past bookings and patterns to suggest slots that match their habits. The result is suggestions that are personalized and contextual, which we found meaningfully improves how often users end up completing a booking rather than bouncing.

D. Production-Grade Infrastructure

Running a booking platform in production means things will break — and you need to know before your users do. We set up Sentry across both frontend and backend so errors get caught and surfaced immediately. Upstash Redis handles caching for frequently accessed data, keeping response times tight. BullMQ pushes email delivery into background workers so it never blocks the main request flow. Payment endpoints are protected with Express rate limiting to guard against brute-force attempts. We also added gzip compression to cut down response sizes and log per-request performance metrics. Health check routes at /health and /ping let uptime monitors do their job without any extra setup.

III. Tech Stack

A. Frontend (Web)

On the web side, we went with React 18 using hooks and context for state management, TypeScript throughout for type safety, and Vite as the build tool because of how fast the dev experience is. Tailwind CSS v4 handles styling with a utility-first approach. For animations, we used both Framer Motion and GSAP — Framer Motion for component transitions and GSAP for scroll-driven effects. React Router v7 manages routing with protected routes where needed. Recharts powers the interactive charts on the owner dashboard, and Zod handles runtime schema validation on form inputs and API responses.

B. Backend

The backend is a Node.js/Express REST API. We use the Firebase Admin SDK for server-side Firestore writes — this lets us bypass client-side security rules when needed, which matters for operations like confirming bookings. The Razorpay SDK handles payment order creation and the HMAC signature verification step. BullMQ with ioredis manages our email queue in the background, and Upstash Redis sits in front of frequently read data for caching. Sentry catches anything that goes wrong in production. The Groq SDK drives AI recommendations, and transactional emails are built with Brevo and React Email templates. Express-rate-limit guards payment endpoints.

C. Mobile

The mobile app is built with Flutter and Dart, which gave us a single codebase for both iOS and Android without sacrificing too much on native feel. Firebase handles auth, Firestore data access, and file storage on the mobile side. Feature-wise, the app covers everything a player needs — searching for turfs, booking slots, and managing their profile.

D. Infrastructure

Both frontend and backend are deployed on Vercel as serverless functions — no server management, and deployments happen straight from the repo. Firebase covers authentication, the Firestore database, file storage, and security rules. For local development we included a Docker Compose setup so the whole stack can spin up consistently across machines. Upstash manages the Redis instance that backs both the caching layer and BullMQ's job queue.

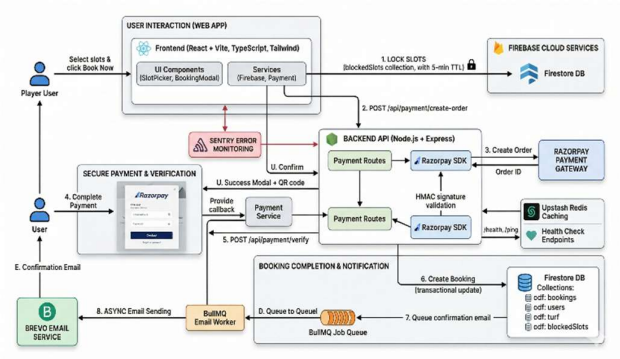
IV. Architecture

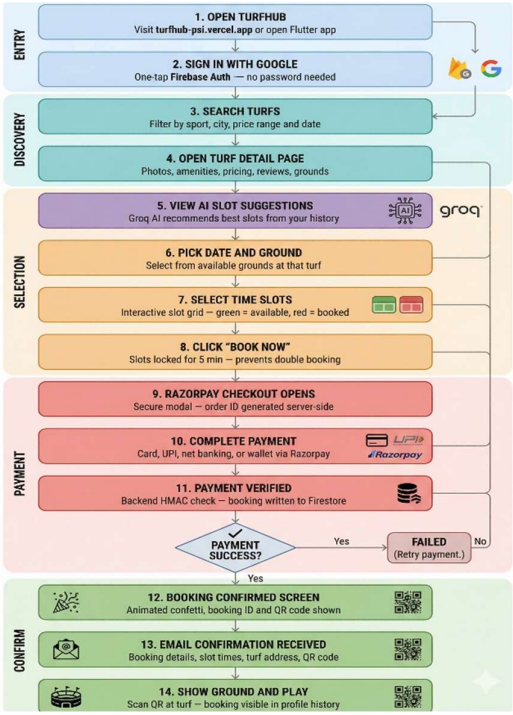
At its core, aLiveHub follows a layered client-server architecture. The client layer has two surfaces: the React/Vite web app and the Flutter mobile app. On the server side, the backend API is organized into five route groups — authentication, payment, turf, admin, and recommendations — each with a clear responsibility boundary.

The web client talks to all five route groups. The mobile client, by contrast, connects directly to Firebase Auth and Firestore for most operations. On the backend, auth routes go through Firebase Auth; payment routes touch both Razorpay and Firestore; turf routes pull from Firestore with Redis caching in front; admin routes write to Firestore; and recommendation routes call Groq AI with results cached in Redis. When a payment succeeds, the payment route drops a job into BullMQ which a background worker picks up to send the confirmation email through Brevo. Sentry watches everything across the entire backend.

A. Booking Request Flow

Here is what happens when someone actually books a slot. The moment they select their time slots, the frontend writes those slots to the blockedSlots collection in Firestore - this creates a 5-minute lock so nobody else can claim them. Then a POST to /api/payment/create-order goes out to Razorpay, which creates the order. The user pays through Razorpay's checkout. Once payment completes, the callback hits /api/payment/verify — we check the HMAC signature to confirm it is legitimate, then write the booking to Firestore inside a transaction and queue the confirmation email. The frontend gets the confirmation back and shows the success modal with the QR code. Separately, the email worker picks up the queued job and delivers the email.

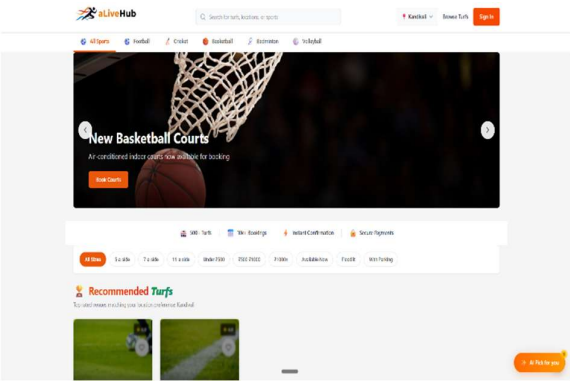


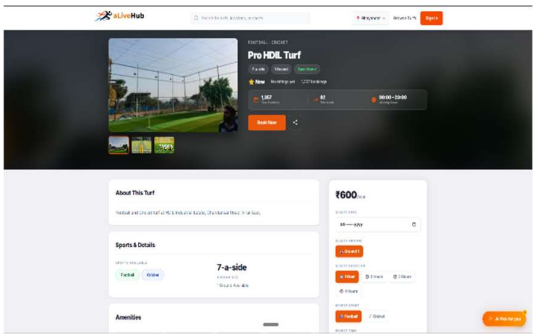


V. Project Structure

The repo splits into three main directories. The frontend directory holds the React/TypeScript app, organized by purpose — feature components like BookingModal, SlotPicker, SmartRecommendation, and HeroCarousel; layout components like Header, Footer, and Navigation; and lower-level UI primitives. Pages are their own layer: Home, TurfListings, TurfDetailPage, the Owner dashboard, Profile, and SignIn all live there.

The backend directory follows a similar logic — config for Firebase Admin and Redis, controllers that handle request logic, middleware for auth and caching, route files (authRoutes, paymentRoutes, turfRoutes, adminRoutes, recommendationRoutes), and service modules for booking, payment, email, and auth. The mobile directory is organized into Flutter screens, data models, API and Firebase service layers, reusable widgets, and theme config. At the root level sit the shared config files: docker-compose.yml, firestore.rules, firestore.indexes.json, and firebase.json.





VI. Quick Start and Deployment

A. Prerequisites

To run the project, you'll need Node.js v18 or above, npm v9 or above, Flutter v3.x if you're working on the mobile app, and Docker if you want to use the containerized setup.

B. Manual Setup

Clone the repo, go into the backend directory, install dependencies, and fill out the .env with your API keys. Start the backend dev server on port 5000. The frontend setup is the same pattern — add your Firebase and Razorpay keys to its own .env and it runs on port 5173. For the Flutter app, run flutter pub get first, then launch normally.

C. Docker Setup

If you want the quickest path to a working local environment, use Docker Compose — it brings up everything including a Firebase emulator in one command. Clone the repo, fill in the backend .env, then run docker-compose up --build. That gives you the frontend on port 5173, the backend API on port 5000, and the Firebase Emulator UI on port 4000.

D. Vercel Deployment

Production deployment is straightforward — run vercel --prod from each directory. The backend's vercel.json points to src/index.js as the entry point using the @vercel/node builder and routes all incoming requests to that handler. The live frontend is deployed at https://alivehub.vercel.app

VII. Environment Variables

A. Backend Variables

On the backend, the variables you must have are GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET for OAuth, JWT_SECRET for signing tokens, RAZORPAY_KEY_ID and RAZORPAY_KEY_SECRET for payment processing, and FIREBASE_PROJECT_ID. Beyond those, several features are gated behind optional variables: UPSTASH_REDIS_REST_URL and REST_TOKEN enable caching, REDIS_URL is needed for BullMQ, GROQ_API_KEY turns on AI recommendations, SENTRY_DSN enables error tracking, and ADMIN_SECRET protects the Bull Board dashboard.

B. Frontend Variables

The frontend needs VITE_API_URL to know where to send requests, plus seven Firebase config keys: VITE_FIREBASE_API_KEY, VITE_FIREBASE_AUTH_DOMAIN, VITE_FIREBASE_PROJECT_ID, VITE_FIREBASE_STORAGE_BUCKET, VITE_FIREBASE_MESSAGING_SENDER_ID, VITE_FIREBASE_APP_ID, and VITE_RAZORPAY_KEY_ID. VITE_SENTRY_DSN is optional but recommended for production.

VIII. API Reference

A. Authentication and Turfs

Auth has two public endpoints: GET /api/auth/google kicks off the Google OAuth flow, and GET /api/auth/google/callback handles the response from Google once the user authenticates. Turf data is also publicly accessible — GET /api/turfs returns all listed turfs, and GET /api/turfs/:id fetches a single turf by ID.

B. Payments and Admin

Payment endpoints require Firebase authentication. POST /api/payment/create-order creates a Razorpay order, POST /api/payment/verify validates the payment signature and writes the booking, and POST /api/payment/cancel handles cancellations. The admin endpoints require owner-level auth: GET /api/admin/dashboard returns booking and revenue statistics, PUT /api/admin/turf/:id updates turf details, and GET /api/admin/bookings retrieves the owner's booking list.

C. AI Recommendations and Health

GET /api/recommendations is Firebase-authenticated and returns AI-generated slot suggestions for the current user. For uptime monitoring, GET / returns basic API status, GET /health gives a detailed health check with service states, and GET /ping is a lightweight endpoint for external monitors.

IX. Database Schema

A. Firestore Collections

Firestore is organized into six top-level collections. The turf collection holds all turf data — name, city, address, type, price per hour, rating, amenities, images, grounds, and owner contact — plus a nested reviews subcollection for user feedback. The bookings collection records everything tied to a confirmed booking: user and turf identifiers, bookedDate, timeSlots, totalPrice, sport, groundId, a status field that is either confirmed or cancelled, payment identifiers, and timestamps.

The users collection keeps name, email, photoURL, phone, favorites, and createdAt for each user. The owners collection links turfId and businessName to an isApproved flag and a createdAt timestamp. blockedSlots is a short-lived collection — each document has turfId, date, slot, blockedBy, and an expiresAt TTL field, and exists only to hold the 5-minute lock that prevents double bookings during checkout. The payments collection is minimal by design: orderId, amount, status, and createdAt.

B. Security Rules

Turf documents are publicly readable, but only a verified owner can update their own turf. Bookings are always written through the Admin SDK on the server — clients cannot write to this collection directly, and users can only read their own booking records. The same pattern applies to user profiles: read and write access is limited to the owner of that profile. For owner accounts, anyone can create a profile, but self-approval is explicitly blocked — only an administrator can flip the isApproved flag. The blockedSlots collection can be managed by the user who created the lock; owners can also clear stale locks left over from abandoned checkouts. Payment documents never touch the client — all access is server-side only.

X. Monitoring and Observability

For observability, we use four tools across different layers. Sentry covers error tracking and performance tracing on both frontend and backend — everything surfaces in the sentry.io dashboard. Vercel Analytics tracks web vitals and speed insights without any extra instrumentation needed. Bull Board sits at /admin/queues (behind authentication) and gives visibility into the BullMQ job queue — useful for debugging email delivery issues. Upstash Console at console.upstash.com shows Redis usage metrics and command stats.

XI. Conclusion

aLiveHub shows that a well-structured full-stack SaaS architecture can genuinely solve a fragmented, real-world problem — in this case, sports turf booking in India. Bringing together a React/TypeScript frontend, a Node.js/Express backend, Firebase Firestore as a scalable NoSQL store, and a Flutter mobile app gave us a cross-platform experience that works for both players looking to book quickly and owners trying to run their business properly.

Integrating Razorpay with server-side HMAC verification kept payments secure without adding friction to the user experience. The 5-minute slot-locking mechanism turned out to be one of the more important design decisions — it is a simple idea but it completely eliminates double bookings. Groq-powered recommendations improved how engaged users were with available slots, and the observability stack — Sentry, BullMQ monitoring via Bull Board, Upstash Redis, and Vercel Analytics — gave us confidence that the system behaves reliably under real load. There is still a lot we want to build: dynamic AI-driven pricing, multi-sport and multi-city expansion, content moderation for reviews, and bringing the full owner dashboard into the mobile app are all on the roadmap.

Acknowledgment

We would like to thank Vidyavardhini's College of Engineering and Technology (VCET), Vasai, affiliated to the University of Mumbai, for the infrastructure, resources, and academic environment that made this project possible. We are grateful to the Department of Information Technology for the encouragement and guidance throughout. Special thanks to our project supervisor Ms. Madhavi Waghmare, whose mentorship, technical input, and feedback shaped the direction of this work in meaningful ways. We also thank our fellow students and colleagues ; the conversations, reviews, and shared effort made a real difference in getting this across the finish line.

References

[1] "TurfIt: An Instant Turf Booking Application," *IEEE Xplore*, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/10927753/

[2] "Sport Field Reservation Based on Mobile Application," *IEEE Conference Publication*, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9307545/

[3] "Turf Tracker Application," *IJRASET*, 2023. [Online]. Available: https://www.ijraset.com/research-paper/turf-tracker-application

[4] "Online Sports Turf Booking System," *IJSREM*, 2023. [Online]. Available: https://ijsrem.com/download/online-sports-turf-booking-system/

[5] "Sports Club Booking Community Web Application," *IRJAET*, 2025. [Online]. Available: https://www.researchgate.net/publication/391466399

[6] "ONETAP: A Unified Multi-Domain Ticket Reservation System," *IJCRT*, 2025. [Online]. Available: https://www.ijcrt.org/papers/IJCRT25A1136.pdf

[7] "BOOKAZOR – An Online Appointment Booking System," *IEEE Xplore*, 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8899460/